Universidad Nacional Autónoma de México
Facultad de Ingeniería
INGENIERÍA EN COMPUTACIÓN

Aprender del error:
El gradiente y la matriz jacobiana

Cardoso Jaramillo Hanniel · Estrada Santos Angel Joseph

Cálculo Vectorial · Grupo 13 · Semestre 2027-1

TEORÍA DE CÁLCULO VECTORIAL

La teoría del aprendizaje

Un modelo produce una predicción $\hat{y}$. Sus **pesos w** determinan la influencia de cada entrada. Al entrenarlo, se compara $\hat{y}$ con un valor conocido **y** y se ajustan los pesos para reducir una **pérdida L**, que cuantifica el error. [1]

1. El gradiente orienta el ajuste

Las derivadas parciales miden el cambio de L al variar un peso y fijar los demás. El **gradiente** las reúne:

$$\nabla_w L = \left[\frac{\partial L}{\partial w_1}, \ldots, \frac{\partial L}{\partial w_p}\right]^T$$

Apunta al mayor crecimiento local de L. Para reducirla, se avanza en dirección opuesta:

$$\mathbf{w^{nuevo} = w - \eta \nabla_w L}$$



η es la **tasa de aprendizaje**: un paso excesivo puede aumentar L.

2. La jacobiana mide sensibilidades

Para $\mathbf{f}: \mathbb{R}^p \to \mathbb{R}^m$, con p pesos y m salidas, la jacobiana organiza las sensibilidades: una fila por salida y una columna por peso:

$$J_w f = \left[\frac{\partial f_i}{\partial w_j}\right]_{m \times p}$$

La entrada (i, j) mide el cambio de la salida i al variar el peso j. Aquí se predice un solo precio: la matriz será de **1 × 3**. [2, 3]

$$\nabla_w L = (J_w f)^T \nabla_{\hat{y}} L$$

La **regla de la cadena** une sensibilidades y error. Aplicarla hacia atrás entre capas calcula los gradientes mediante **retropropagación**. [1–3]

Aplicación en nuestra ingeniería

En Ingeniería en Computación se programan modelos y derivadas automáticas para analizar imágenes, lenguaje y series temporales. Aquí usamos **una neurona lineal sin sesgo** para desarrollar a mano una iteración de entrenamiento. [1–3]

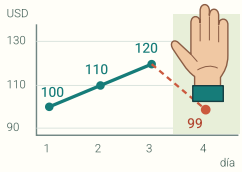
Conclusiones

La jacobiana relaciona pesos y predicción; el gradiente orienta su ajuste. La pérdida baja **59.68 %** en este caso. Para evaluar la generalización se requieren **datos que no se usaron para entrenar**.

EJEMPLO ESTRELLA · UNA ITERACIÓN COMPLETA

Tapar, predecir, destapar… y corregir los pesos

Imagina que tapas con una mano el último tramo de una gráfica de NVIDIA. El modelo recibe los tres primeros precios y predice el cuarto; después quitas la mano y comparas. Usaremos **precios hipotéticos, en USD**: $x = (100, 110, 120)^T$ y un objetivo oculto $y = 135$.



La mano oculta la respuesta.
Los días 1–3 son las entradas.

1 Predecir sin ver el cuarto día

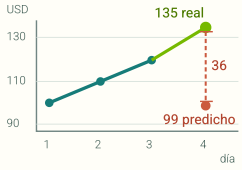
Partimos de $w^{(0)} = (0.3, 0.3, 0.3)^T$. Cada peso multiplica un precio; luego se suman los aportes.

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3$$

$$\hat{y}_0 = 0.3(100) + 0.3(110) + 0.3(120)$$

$$\hat{y}_0 = 30 + 33 + 36 = \mathbf{99\ USD}$$

Los pesos no tienen que sumar 1: el modelo no es un promedio.



Quitamos la mano: y = 135.
La predicción quedó 36 USD abajo.

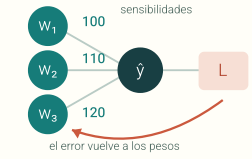
2 Destapar y medir el error

Revelamos el valor conocido. El error tiene signo; la pérdida cuadrática permite medir su magnitud.

$$e_0 = \hat{y}_0 - y = 99 - 135 = \mathbf{-36\ USD}$$

$$L(w) = \tfrac{1}{2}(\hat{y} - y)^2$$

$$L_0 = \tfrac{1}{2}(-36)^2 = \tfrac{1}{2}(1296) = \mathbf{648\ USD^2}$$



Del resultado a los pesos.
Las derivadas reparten el ajuste.

3 Calcular jacobiana y gradiente

Con x fijo, $\partial\hat{y}/\partial w_i = x_i$. La jacobiana es de 1 × 3:

$$J_w\hat{y} = [100 \quad 110 \quad 120]$$

Por la regla de la cadena, para cada i = 1, 2, 3:

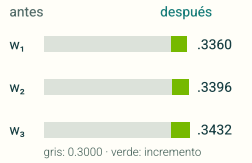
$$\frac{\partial L}{\partial w_i} = \tfrac{1}{2} \cdot 2(\hat{y} - y)\frac{\partial \hat{y}}{\partial w_i} = (\hat{y} - y)x_i$$

$$\partial L/\partial w_1 = (-36)(100) = -3600$$

$$\partial L/\partial w_2 = (-36)(110) = -3960$$

$$\partial L/\partial w_3 = (-36)(120) = -4320$$

$$\nabla_w L = \mathbf{(-3600, -3960, -4320)^T}$$



Se resta un gradiente negativo:
los tres pesos aumentan.

4 Actualizar los tres pesos

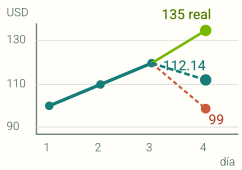
Elegimos η = 0.00001. Todos los ajustes usan el mismo gradiente calculado en $w^{(0)}$.

$$w^{(1)} = w^{(0)} - \eta \nabla_w L$$

$$w_1^{(1)} = 0.3 - 0.00001(-3600) = \mathbf{.3360}$$

$$w_2^{(1)} = 0.3 - 0.00001(-3960) = \mathbf{.3396}$$

$$w_3^{(1)} = 0.3 - 0.00001(-4320) = \mathbf{.3432}$$



Verde: valor real · **rojo:** inicial
Azul verdoso: predicción ajustada.
Se evalúa el mismo caso de entrenamiento.

5 Comprobar cuánto mejoró

$$\hat{y}_1 = .3360(100) + .3396(110) + .3432(120)$$

$$\hat{y}_1 = 33.600 + 37.356 + 41.184 = \mathbf{112.14\ USD}$$

$$e_1 = 112.14 - 135 = -22.86\ \text{USD}$$

$$L_1 = \tfrac{1}{2}(-22.86)^2 = \mathbf{261.2898\ USD^2}$$

$$\text{Reducción} = \frac{648 - 261.2898}{648} \times 100 = \mathbf{59.68\ \%}$$

648 → 261.29 La pérdida disminuye.
La respuesta aún no es exacta.

BIBLIOGRAFÍA Y MESOGRAFÍA

[1] Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). *Learning representations by back-propagating errors.* Nature, 323, 533–536. doi:10.1038/323533a0.

[2] Baydin, A. G. et al. (2018). *Automatic Differentiation in Machine Learning: a Survey.* JMLR, 18(153), 1–43. jmlr.org/papers/v18/17-468.html.

[3] PyTorch. *Automatic Differentiation with torch.autograd.* PyTorch Tutorials, documentación oficial.

Diagramas propios; escudos del boceto original; logotipo de NVIDIA, pautas oficiales de marca. Consulta: 08/11/2026.

Caso didáctico: los precios fueron inventados para mostrar el procedimiento. El logotipo identifica la empresa del ejemplo; no implica afiliación ni patrocinio.